

Contents

Livermore Strategy Builder — Deep Dive: Modules, Flows, and the Quant Expert’s Map 1

Table of contents	1
1. Orientation	1
2. Module inventory	2
3. Flow A: “Use a Template”	13
4. Flow B: “Custom Build”	15
5. Quant expert’s technical map — have vs need	16
6. Maturity positioning	20
7. Cross-references	20

Livermore Strategy Builder — Deep Dive: Modules, Flows, and the Quant Expert’s Map

What this doc is: A granular module-by-module read of the strategy-builder system, paired with the two end-to-end product flows narrated step by step, framed against a quant trading expert’s checklist of what a serious strategy platform needs.

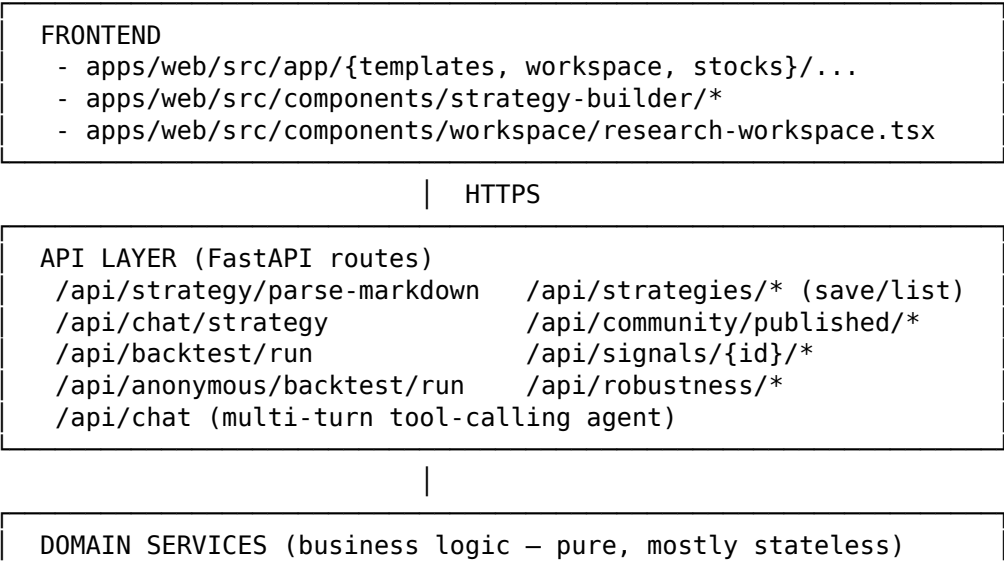
Companion to: `strategy_builder_audit.md` (high-level scorecard).

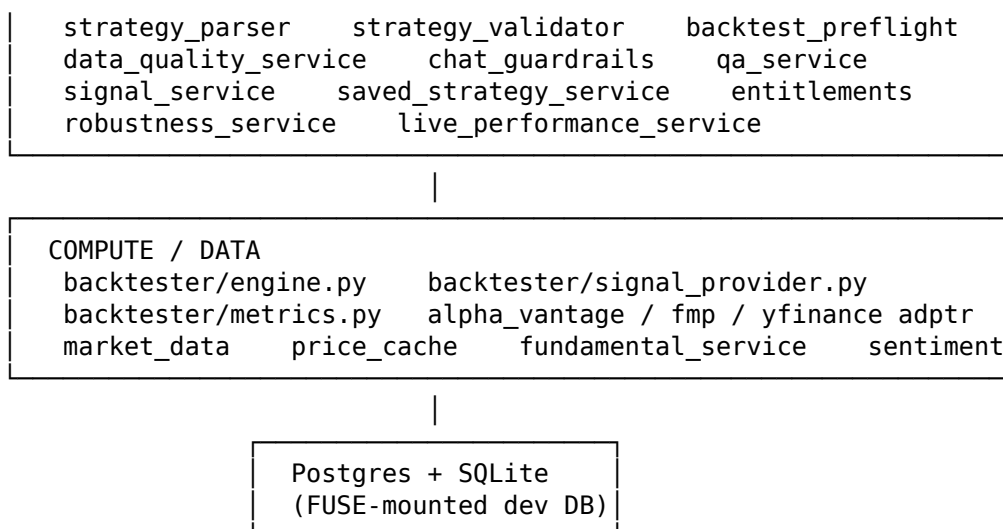
Table of contents

- 1. [Orientation — how the system is wired](#)
- 2. [Module inventory — what’s enabled, what’s built](#)
- 3. [Product flow A: “Use a Template”](#)
- 4. [Product flow B: “Custom Build”](#)
- 5. [The quant expert’s technical map — have vs need](#)
- 6. [Where Livermore sits on the maturity curve](#)

1. Orientation

The Livermore strategy-builder is split across **3 backend layers** and **2 frontend layers**:





Two key architectural decisions to keep in mind:

- **Backtester is deterministic.** Same StrategyJSON → same result, byte-for-byte. The LLM only parses input and explains output; it never decides what to trade.
- **Tier gating is centralized.** All entitlement caps (universe size, history window, runs/week, save count) live in `services/entitlements.py:TIER_CAPS` and are enforced at the route layer via `deps_entitlement`. Templates are exempt from universe/history caps; only custom strategies are gated.

2. Module inventory

Organized by what a quant strategy platform actually needs. Each module rated:

- ☒ **Shipped** — production code, tested, in use.
- ☐ **Partial** — exists but limited or stub.
- ☐ **Missing** — not built.

2.1 Data layer

Module	Path	Status	What it does
Alpha Vantage adapter (EOD prices)	<code>services/alpha_vantage.py</code>	☒	Daily OHLCV bars, splits/dividends-adjusted. Default historical source.
FMP adapter	<code>services/adapters/fmp_adapter.py</code> , <code>services/fmp_client.py</code>	☒	Live quotes, fundamentals (income statement, balance sheet, cash flow), earnings dates/estimates, insider transactions, news.
yfinance adapter	<code>services/adapters/yfinance_adapter.py</code>	☐	Fallback / supplemental data source.

Module	Path	Status	What it does
Price cache (DB-backed)	services/price_cache_service.py	☑	Local persistence of bars per symbol; ensure_history(symbol, from_date) lazy-loads.
Live quote cache	services/live_quote_service.py	☑	30s in-process TTL cache, per-symbol asyncio locks (thundering-herd protection). Used by ticker bar, stock detail, workspace, market pulse, community feed.
News articles cache	services/news_article_cache_service.py	☑	
SEC EDGAR client	services/sec_edgar_client.py	☑	Insider Form-4 filings.
Symbol registry	services/symbol_service.py	☑	Local symbol table + Alpha Vantage search fallback.
Corporate actions integrity	—	☐	Adjusted close present, but no explicit split/dividend reconciliation pipeline. Relies on vendor adjustments.
Point-in-time index membership	app/data/sp500_ticker.py	☑	Current-membership only. Historical S&P 500 membership not tracked — survivorship bias is acknowledged in chat_guardrails but not architecturally prevented for cross-sectional templates.
Survivorship-bias-free universe	—	☐	No “alive as of date X” universe service.
Macro data feed (rates, inflation, dollar)	—	☐	Not built.
Options data	—	☐	Not in scope yet.

2.2 Universe definition

Module	Path	Status	What it does
Static universe list (S&P 500)	app/data/sp500_ticker.py	☑	Hardcoded ticker list.
Custom ticker input	components/universe-input.tsx	☐	Comma-separated input with validation.

Module	Path	Status	What it does
Universe presets in custom builder	strategy-builder-modal.tsx:DEFAULT_CUSTOM.universe	☐	“sp500” / “nasdaq100” / “russell1000” / “custom” / “skip”.
Universe size cap by tier	services/entitlements.py:TIER_CAPS	☐	Scout: 5; Strategist: 25; Quant: 100 (custom strategies). Templates exempt.
Dynamic screener (filter by mcap, liquidity, sector)	services/screener_service.py	☐	screen(filters) returns filtered universe. Not yet wired into the strategy builder.
Point-in-time universe membership	—	☐	Backtests against the current S&P 500 list, not the as-of-date list.
Industry / sector classification at backtest time	—	☐	Available via competitor_group_service.py and value_chain_classifier.py but not integrated into strategy filters.

2.3 Signal engine

Module	Path	Status	What it does
Price-based primitives (inline in engine)	services/backtester/engine.py:_generate_weights	☐	MA filter, MA crossover, breakout, RSI, Bollinger MR — each is a hardcoded branch.
Cross-sectional ranking helper	services/backtester/engine.py:_generate_cross_sectional_weights	☐	Consent top-N weights top-pct ranker; shared by 12 strategy types.
Fundamental signal provider	services/backtester/signal_provider.py:FundamentalSignalProvider	☐	Discard stale book/market, EBITDA/EV, buyback yield, estimate revisions. Point-in-time enforced via disclosure-date lag (default 45 days).
Sentiment signal provider	services/backtester/signal_provider.py:SentimentSignalProvider	☐	30-day rolling FinBERT sentiment per ticker.
Earnings event provider	services/backtester/signal_provider.py:EarningsEventSignalProvider	☐	Uses event date, NaN elsewhere.
Insider signal provider	services/backtester/signal_provider.py:InsiderSignalProvider	☐	Custom buy/selling buying signal.

Module	Path	Status	What it does
Signal provider registry	services/backtester/signal_provider.py	get_signal_provider(name)	dispatch.
Signal composition (logic_with_prior)	—	□	Schema doesn't carry AND/OR-with-prior-rule. Multi-rule strategies use implicit AND within a strategy_type but can't be composed by the user.
Price-based primitives as SignalProviders	—	□	They're inline in _generate_weights rather than wrapped in the provider protocol. Refactor needed to make them composable with fundamental/sentiment.
Per-primitive fire-rate counting	—	□	Engine doesn't track per-primitive truth values per bar.
Contribution attribution	—	□	No "which primitive triggered this trade" record.
Custom user-defined primitives (Python sandbox)	—	□	No support for Pro-tier user-defined signal functions.

2.4 Strategy definition language (DSL)

Module	Path	Status	What it does
StrategyJSON schema	apps/api/app/schemas/strategy.py	5	22 strategy_type literals; nested StrategyRule[], PositionSizing, RiskManagement, CashManagement. Pydantic v2 with model validators.
Frontend mirror types	apps/web/src/lib/contracts.ts	StrategyJson	TypeScript mirror; manual sync with backend (typed test in tests).
Markdown → strategy parser	services/strategy_parser.py	5	parse_strategy_markdown research note → structured StrategyJSON; emits ambiguities and assumption_log.

Module	Path	Status	What it does
Chat → strategy parser	services/strategy_parser.py	parse_strategy_Natural	language → StrategyJSON with multi-turn previous_strategy_json support. LLM + deterministic fallback.
Strategy validator	services/strategy_validator.py		Tier-A warnings: short backtest window, no costs, MA cross direction, vol_target without target. No semantic consistency check (entry trend + exit MR).
Versioning / immutable strategy snapshots	models/saved_strategy.py		Saved strategies are immutable per save, but no “version 1, 2, 3” lineage.

2.5 Backtest engine

Module	Path	Status	What it does
BacktestEngine.run()	services/backtester/engine.py		Loads prices, builds weights, computes returns, captures trade log, persists BacktestRecord.
22 strategy-type branches	same	□	All strategy types in the schema have engine implementations.
Equal-weight & fixed-weight sizing	same	□	
Vol-target sizing overlay	same (line 702)	□	Scales weights to hit target_vol_annual, capped at 1.0 (no leverage).
Signal-weighted sizing	same	□	Schema has the literal but engine raises NotImplementedError.
Transaction-cost model	same	□	(transaction_cost_bps + slippage_bps) × turnover applied per-bar.
Fills	same	□	Next-bar close approximation; rebalances applied on rebalance dates.

Module	Path	Status	What it does
Lookback warmup auto-sizing	<code>_compute_lookback</code>	☐	Pre-start window trimmed before P&L computation.
Trade log extraction	<code>_extract_trade_log</code>	☐	Per-symbol entry/exit, return, holding period.
Multi-asset benchmark comparison	same	☐	SPY-default; configurable.
Buy-and-hold baseline (single-asset)	same	☐	
Borrow cost for shorts	—	☐	Engine is long-only; no negative-weight support.
Funding cost for leverage	—	☐	Same — no leverage path.
Realistic intraday execution	—	☐	EOD bars only.

2.6 Risk management

Module	Path	Status	What it does
Per-position stop loss	<code>schemas/strategy.py:RiskManagement.stop_loss + engine</code>	☐	Engine watches per-position drawdown from entry.
Take-profit	<code>RiskManagement.take_profit_pct + engine</code>	☐	
Portfolio max-drawdown stop	<code>RiskManagement.max_drawdown_stop + engine</code>	☐	Latches to 100% cash once peak-to-trough exceeds threshold.
Hold-cash-when-no-signal	<code>CashManagement.hold_cash_when_no_signal</code>	☐	Default behavior; configurable.
Cash yield (idle capital)	<code>CashManagement.cash_yield_bps</code>	☐	
Risk preset (low/medium/high) → engine params	<code>lib/strategy-picker/risk-presets.ts:applyRiskLevel</code>	☐	Frontend helper. Maps preset → <code>vol_target + max_drawdown_stop + stop_loss_pct</code> .
Position concentration cap	—	☐	<code>PositionSizing.max_positions</code> limits count; no explicit max-position-pct enforced at the engine layer (relies on equal-weight semantics).
Sector / factor exposure caps	—	☐	Not modeled.

Module	Path	Status	What it does
Correlation-aware sizing	—	☐	
Trailing stop	—	☐	Stop loss is from entry, not from running peak.

2.7 Performance & metrics

Module	Path	Status	What it does
compute_metrics (25+ metrics)	services/backtester/metrics.py	☐	Total return, annualized return, vol, Sharpe, Sortino, max drawdown, Calmar, win rate, profit factor, avg winner/loser, median trade return, longest win/lose streak, beta/alpha vs benchmark, turnover, time-in-market.
Trade diagnostics	same	☐	
Buy-and-hold baseline metrics	same	☐	
Equity curve + drawdown curve	engine output	☐	
Annual / monthly returns	engine output	☐	Monthly heatmap in workspace.
Benchmark comparison	engine output	☐	Computes excess return vs benchmark (default SPY).
Credibility warnings	routes/backtest.py:credibility_warnings	☐	Flags Sharpe > 2.0, win rate > 80% with ≥10 trades, total return > 100% in < 1 yr.
Factor attribution	—	☐	No factor decomposition (value/momentum/quality contribution).
Per-asset attribution	—	☐	Trade log gives per-symbol P&L but no contribution view.
Information ratio, hit rate by regime	—	☐	

2.8 Robustness testing

Module	Path	Status	What it does
RobustnessService	services/robustness_service.py	☐	Full async robustness suite.

Module	Path	Status	What it does
Parameter sensitivity	RobustnessService.run_parameter_sensitivity		Grid search over per-strategy-type parameter ranges (e.g. MA lookback 50–300). Returns Sharpe/return/MaxDD heatmap.
Sub-period analysis	RobustnessService.run_subperiod		Splits backtest window into chunks; reports stability of returns.
Transaction-cost sensitivity	RobustnessService.run_transaction_cost		Re-runs at cost levels [0, 5, 10, 25, 50] bps.
Benchmark comparison	RobustnessService.run_benchmark_comparison		Compares to SPY / QQQ / IWM.
Peer-ticker comparison	RobustnessService.run_peer_ticker		Same strategy on alternate tickers in same sector.
Frontend robustness tab	workspace/research-workspace.tsx Robustness tab	☐	Renders all five panels.
Walk-forward optimization	—	☐	Sub-period is close but not true walk-forward (no parameter re-fitting per window).
Out-of-sample testing	—	☐	No automated train/test split.
Monte Carlo simulation	—	☐	
Regime conditioning	—	☐	No bull/bear regime-specific stats.
Capacity analysis	—	☐	No AUM-vs-slippage curve.

2.9 Strategy lifecycle management

Module	Path	Status	What it does
Save strategy	services/saved_strategy_service.py:save_strategy	☐	Fit to Strategy JSON + visibility. Tier-capped (Scout: 10).
List user's saved strategies	same	☐	
Get / delete saved strategy	same	☐	
Visibility toggle (public/private)	routes/strategy_storage.py:PATCH /{slug}/visibility	☐	Scout auto-public.
Comparison view (current vs saved)	workspace Comparison tab	☐	Side-by-side metrics.
Forking	models/saved_strategy.py:forked_from_id (FK)		Saved strategies have a forked_from reference.

Module	Path	Status	What it does
Publish to community	services/community_publish_service.py, routes/community_publish.py	✓	Publish → feed entry with author badge + summary.
Public strategy detail page	app/s/[slug]/page.tsx	✓	
Live performance tracking	app/strategies/[slug]/page.tsx services/live_performance_service.py	✓	Daily cron updates published strategies' trailing returns.
Strategy versioning (v1, v2, v3 history)	—	□	Each save is immutable but there's no lineage thread. Fork is the workaround.

2.10 Live deployment / paper trading

Module	Path	Status	What it does
Live signal computation	services/signal_service.py	✓	Cron computes current signal as of date for each saved strategy. Classifies changes.
Signal event log	models/signal_event.py (signal_events table)	✓	Append-only; one row per signal change.
Signal state cache	models/saved_strategy_signal_state.py	✓	Latest signal per strategy.
Signal alert subscriptions	models/signal_alert_subscription.py	✓	User opt-in per saved strategy.
Email signal alerts	emails/signal_alert.py + jobs/signal_jobs.py	✓	Cron-dispatched email when signal flips.
Signal endpoint (frontend pull)	routes/signals.py	□	GET/POST/DELETE/acknowledge.
Signal history drawer	frontend	□	Endpoint exists; UI partial.
Broker execution (paper or live)	—	□	No order routing layer.
Position reconciliation	—	□	Live signals tell users what to trade; doesn't track what they actually did.

2.11 User interface

Module	Path	Status	What it does
Template gallery (/templates)	app/templates/page.tsx	✓	Filter by category, evidence tier, capacity, horizon. 25 templates.

Module	Path	Status	What it does
Per-template detail page	app/templates/[slug]/page.tsx	🟢	Read-only with “Use this template” CTA.
Strategy builder modal	components/strategy-builder/strategy-builder-modal.tsx	🟢	Step machine; both paths converge here.
5-question wizard	components/strategy-builder/wizard/strategy-wizard.tsx	🟢	One-question-at-a-time with animated transitions.
4-block summary step	components/strategy-builder/summary-step.tsx	🟢	WHAT / WHEN IN / HOW MUCH / WHEN OUT.
Workspace result viewer	components/workspace/research-workspace.tsx	🟢	Tabs: Detailed Metrics, Review, Robustness, History.
Equity / drawdown charts	components/workspace/charts.tsx	🟢	
Monthly return heatmap	components/workspace/monthly-heatmap.tsx	🟢	
Stock detail page (/stocks/[ticker])	app/stocks/[ticker]/page.tsx	🟢	Has business-model, sentiment, evaluation, market-position sections.
Commodities pages	app/commodities/*	🟢	
Community feed (/community)	app/community/page.tsx	🟢	Browse published strategies.
Chat widget	components/ChatWidget.tsx	🟢	Multi-turn agent with 7 tools.
WHEN IN / WHEN OUT primitive composer	—	🟢	The block UI shows template-baked copy; not editable.
Asset behavior fingerprint UI (“trends 65% of the time”)	—	🟢	
Per-primitive fire-rate panel in results	—	🟢	

2.12 Quality gates & safety

Module	Path	Status	What it does
Universe existence check	services/backtest_preflight.py:validate_universe	🟢	Symbol service + Alpha Vantage search; rejects unknown tickers.
Data availability gate	services/backtest_preflight.py:ensure_data_availability	🟢	Lazy-fetches history for symbols with no cached bars.
DataQualityService pre-engine	services/data_quality_service.py	🟢	Blocks backtests on missing data; warns on stale data.

Module	Path	Status	What it does
Credibility warnings post-engine	routes/backtest.py:credibility_warnings		Sharpe > 2 / win > 80% / total > 100% in < 1 yr → warning.
Custom strategy caps (universe, history)	routes/backtest.py:enforce_custom_caps		Tier-gated; shadow-mode logging when GATING_ENABLED=false.
Chat guardrails (refusal, citation)	services/chat_guardrails.py		Refusal classifier (4 categories) + uncited-numeric reprompt (one-shot).
QA review (LLM-as-judge sandbox)	services/qa_service.py		Skeptic that critiques user's backtest result for trustworthiness.
Sentiment sandbox reviewer	services/sentiment_sandbox_reviewer.py		Same pattern, sentiment-specific.
Lookahead detection	—	□	Architectural (signal lag default) but no automated test detecting it on custom strategies.
Survivorship-bias detection	—	□	Acknowledged but not enforced for cross-sectional templates.
Cross-block semantic consistency (entry trend + exit MR warning)	—	□	
Turnover-vs-cost preflight	—	□	

2.13 Telemetry & observability

Module	Path	Status	What it does
PostHog event capture	services/posthog_service.py		backtest_started, backtest_completed with strategy_type + is_template flags.
gate_event shadow-mode logs	routes/backtest.py	□	Structured log lines when caps would have fired but GATING_ENABLED=false.
Refusal / uncited-numeric event logs	services/chat_guardrails.py		Structured logs for nightly QA digest.
Tier weekly usage counter	models/weekly_usage.py + services/entitlements.py		Custom backtests / template backtests / saves.

Module	Path	Status	What it does
template_usage_events—table (spec'd in iteration framework)	—	☐	Mentioned in sql-schema.sql but no SQLAlchemy model yet.
Aggregated per-template analytics in DB	—	☐	PostHog has it; not normalized into a queryable local table for the quarterly cycle.

2.14 Chat agent (cross-cutting)

Module	Path	Status	What it does
Chat endpoint	routes/chat.py	☐	Multi-turn tool-calling.
LLM gateway	services/llm_adapter.py	☐	Provider-agnostic (OpenAI-compatible interface).
Tool: template_search	services/chat_tools/template_search.py	☐	Search the catalog mirror.
Tool: strategy_builder_iterate	services/chat_tools/strategy_builder_iterate.py	☐	Multi-turn refinement.
Tool: backtest_execute	services/chat_tools/backtest_execute.py	☐	Run a backtest from chat; returns headline metrics + backtest_id.
Tool: backtest_explain	services/chat_tools/backtest_explain.py	☐	Fetch full result + LLM explanation.
Tool: concept_explainer	services/chat_tools/concept_explainer.py	☐	Define quant terms.
Tool: onboarding_tutor	services/chat_tools/onboarding_tutor.py	☐	New-user walkthrough.
Tool: stock_lookup	services/chat_tools/stock_lookup.py	☐	Single-stock metadata.
Conversation persistence	models/chat.py	☐	

3. Flow A: “Use a Template”

The path most users will take. Browse-first.

Step-by-step narration

Step 1 — Discovery (entry points). A user arrives at the template gallery via: - Direct /templates URL. - Homepage “Try a template” CTA. - Stock detail page “Build a strategy for AAPL” button (carries ?ticker=AAPL query param). - Chat agent recommendation via template_search tool.

Module touched: app/templates/page.tsx (gallery), lib/contracts.ts:researchTemplates (catalog), chat_tools/template_search.py (chat path).

Step 2 — Filter & select. Gallery shows 25 cards. Filters: category dropdown (“All” / Momentum / Rotation / Factor / Sentiment / etc.), evidence-tier filter, capacity filter, horizon filter, free-text

search. Each card shows badges (A/B/C evidence, capacity, horizon), one-line description, and a “Use template” button.

Status: ☐ shipped.

Step 3 — Template brief. User clicks a template card. Opens /templates/[slug] (full read-only page) or the modal’s template-brief step. Shows: thesis, default settings preview JSON, caveats. CTA: “Customize and run”.

Status: ☐ shipped.

Step 4 — Universe input. User enters their universe (single ticker, or basket). Pre-filled from query param if entering from stock detail. Universe presets (“S&P 500” / “Nasdaq 100” / “Russell 1000”) for cross-sectional templates.

Module: components/universe-input.tsx. **Status:** ☐ shipped. **Missing:** No “asset behavior fingerprint” shown next to the picker — user can’t tell at this step whether the template fits the asset they entered.

Step 5 — 4-block summary review (SummaryStep).

Block	Content	Editable?
1. WHAT	The universe the user just entered	Yes — back button
2. WHEN IN	Plain-English template-baked entry signal	No (read-only)
3. HOW MUCH	Risk preset (Low/Med/High), weight distribution, capital, costs	Yes
4. WHEN OUT	Plain-English template-baked exit signal	No (read-only)

When user picks the risk preset, `applyRiskLevel(strategy, preset)` mutates the `StrategyJSON` to add `vol_target`, `max_drawdown_stop`, `stop_loss_pct`.

Status: ☐ shipped.

Step 6 — Backtest preflight + run. User clicks “Run backtest”. Frontend POSTs to /api/backtest/run. The endpoint: 1. Tier checks → custom-strategy caps don’t apply (templates exempt). 2. `validate_universe` → reject unknown tickers. 3. `ensure_data_available` → lazy-fetch missing price history. 4. `DataQualityService.check_strategy` → block on missing data, warn on stale. 5. `BacktestEngine.run` → produces `BacktestResult`. 6. Append data-quality warnings + credibility warnings. 7. Bump weekly template-backtest counter; emit `PostHog backtest_completed`.

Status: ☐ shipped end-to-end.

Step 7 — Result viewer (workspace). Workspace tabs: - **Detailed Metrics** — equity curve, drawdown curve, monthly heatmap, 25+ metrics, trade log, benchmark comparison, buy-and-hold baseline. - **Review** — LLM explanation + skeptic critique. Citation chips on every numeric claim (enforced by `chat_guardrails`). - **Robustness** — parameter sensitivity, sub-period, transaction cost, benchmark, peer-ticker panels. - **History** — comparison against previously saved strategies.

Status: ☐ shipped.

Step 8 — Save / publish / monitor. User can: - Save the strategy (POST /api/strategies). Scout users auto-public + capped at 10. - Publish to community (POST /api/community/published) → appears in /community feed. - Subscribe to signal alerts (POST /api/signals/{id}/subscribe) → daily cron emails when the signal flips.

Status: ☐ shipped.

What's missing in Flow A

The template flow is end-to-end functional. The gaps are educational/safety polish, not functional blockers:

1. **Asset behavior fingerprint** (“this stock has trended 65% of the time over 10 years”) at Step 4 — would tell the user *before* they run the backtest whether the template fits their asset.
 2. **Soft mismatch warning at preflight** — “You picked an RSI mean-reversion template for NVDA, which has been strongly trending; backtests typically underperform on such assets.”
 3. **Sample-size warning** in results — when trade count < 20, mark Sharpe / win rate as “high variance”.
 4. **template_usage_events analytics table** — for the quarterly cycle’s Step 1 telemetry.
-

4. Flow B: “Custom Build”

The path for users who don’t see a template that fits.

Step-by-step narration

Step 1 — Entry to the wizard. Two ways in: - Modal “Build custom” button → StrategyWizard. - Free-form chat → LLM uses `strategy_builder_iterate` tool to refine across turns → eventually yields a `StrategyJSON`.

Status: ☐ both paths work.

Step 2 — 5-question wizard. Questions, in order: 1. **Asset shape** (single stock / single commodity / stock basket / sector ETFs / broad ETF / pair). 2. **Goal** (growth / defense / alpha / learn / hedge). 3. **Cadence** (daily / weekly / monthly / quarterly / yearly). 4. **Behavior** (trend / mean / unknown). 5. **Drawdown tolerance** (low / medium / high).

Wizard scores 20 strategies; returns top 3 with reasoning.

Module: `wizard/strategy-wizard.tsx`, `wizard/strategy-wizard-data.ts`, `wizard/strategy-wizard-recommend.ts`. **Status:** ☐ shipped. **Missing:** Asset behavior fingerprint (auto-detected from price history) isn’t used to override the user’s behavior answer or to flag inconsistency.

Step 3 — User picks one of the three recommendations. At this point, the picked strategy is *exactly the same shape as a template*. The user is now in the template-summary flow but with a slightly different label.

Status: ☐ shipped.

Step 4 — Summary step. Same `SummaryStep` as Flow A. **And here’s the gap:** the user has just chosen a “custom” strategy but they cannot actually customize WHEN IN or WHEN OUT. Those blocks are still read-only template-baked copy via `template.whenInCopy ?? deriveWhenIn(...)`.

The only editable surface is HOW MUCH (risk preset, weights, capital, costs).

Module: `summary-step.tsx`. **Status:** ☐ partial — flow runs but isn’t truly “custom”.

Step 5 — Same backtest + result flow as Flow A. From here, the path converges with the template flow. Backtest, view results, save, publish.

What's missing in Flow B

The signal-composition layer. Specifically:

1. **Signal primitive library (frontend catalog):** an enumeration of all the atomic building blocks (MA filter, MA crossover, breakout, RSI, Bollinger, return-over-N, vol-over-N, factor rank, F-score, etc.) with their parameter ranges. Doesn’t exist; would live in `lib/signal-primitives.ts`.

2. **WHEN IN composer**: checkbox-style picker of primitives + AND/OR toggle + per-primitive parameter editors. Replaces the read-only block in `summary-step.tsx`.
3. **WHEN OUT composer**: same.
4. **logic_with_prior schema field**: needs to be added to `StrategyRule` so multi-primitive composition can round-trip through the API.
5. **Engine multi-rule fold**: per-symbol boolean strategies need to evaluate each rule into a `Series` and fold with the operator.
6. **Cross-block consistency validator**: “entry trend + exit mean-reversion” → warn.
7. **Turnover-vs-cost preflight**: catch “daily reversal + 5 bps slippage” before the backtest runs.
8. **Custom signal primitives (Pro tier)**: user-defined Python signal functions in a sandbox. Far-future.

When these land, “Custom Build” finally lives up to its name. Today it’s better described as “Pick a template via wizard then tune risk”.

5. Quant expert’s technical map — have vs need

A working checklist of what a serious quant strategy platform requires. Status reflects Livermore’s current state.

Tier 1 — Foundation (no platform without these)

Capability	Status	Notes
Daily OHLCV price data, splits/dividend adjusted	☐	Alpha Vantage + FMP + yfinance
Fundamentals (income, balance, cash flow)	☐	FMP. Point-in-time disclosure-date lag in <code>FundamentalSignalProvider</code> .
Earnings calendar + estimates	☐	FMP
Insider transactions	☐	SEC EDGAR
News sentiment	☐	Alpha Vantage + FinBERT
Symbol resolution + caching	☐	<code>SymbolService</code> + <code>PriceCacheService</code>
Backtest event loop (per-bar)	☐	<code>BacktestEngine.run</code>
Position sizing primitives (equal, fixed, vol-target)	☐	
Transaction cost + slippage model	☐	bps-based, configurable
Risk management (per-position + portfolio stops)	☐	<code>RiskManagement</code> block
Strategy DSL with validation	☐	<code>StrategyJSON</code>
Standard performance metrics (Sharpe, Sortino, DD, etc.)	☐	25+ metrics

Score: 12/12. Foundation is solid.

Tier 2 — Strategy expressiveness (where most platforms compete)

Capability	Status	Notes
Single-asset trend strategies (MA, breakout)	☐	
Single-asset mean-reversion (RSI, Bollinger)	☐	
Cross-sectional momentum	☐	_generate_cross_sectional_weights helper
Sector / asset-class rotation	☐	sector_rotation, dual_momentum
Factor strategies (value, quality, low-vol)	☐	value_composite, quality_piotroski, low_volatility, buyback_yield
Event-driven (PEAD, earnings revision)	☐	Via EarningsEventSignal-Provider
Alt-data signals (sentiment, insider)	☐	SentimentSignalProvider, InsiderSignalProvider
Multi-factor composite	☐	multi_factor_composite strategy_type
Pairs trading (long-only)	☐	pairs_trading strategy_type
Signal composition (AND/OR primitives)	☐	Schema gap + UI gap; biggest missing piece
Long/short equity	☐	No negative-weight support in engine
Options strategies (covered calls, vol)	☐	Out of scope
Intraday signals (ORB, VWAP)	☐	No minute-bar data

Score: 9/13. Excellent for long-only daily-bar equity + ETF + commodities; gaps are in composition flexibility and shorting.

Tier 3 — Robustness & trustworthiness (separates serious from toy)

Capability	Status	Notes
Lookahead-bias prevention	☐	Signal lag default; disclosure-date in fundamentals
Survivorship-bias awareness	☐	Acknowledged but current-membership universes used in backtests
Realistic transaction costs	☐	Configurable per-strategy
Parameter sensitivity testing	☐	RobustnessService.run_parameter_sensitivity
Sub-period analysis	☐	run_subperiod
Transaction cost sensitivity	☐	run_transaction_cost
Benchmark + peer comparison	☐	run_benchmark_comparison, run_peer_ticker
Credibility warnings	☐	Sharpe > 2, etc.
LLM skeptic / sandbox reviewer	☐	qa_service + sentiment_sandbox_reviewer
Walk-forward optimization (parameter refit per window)	☐	Sub-period is close but not true WFO

Capability	Status	Notes
Out-of-sample testing (train/test split)	☐	
Monte Carlo / bootstrap simulation	☐	
Regime conditioning (bull/bear/sideways stats)	☐	
Capacity analysis (AUM vs slippage)	☐	

Score: 9/14. Robustness suite is unusually strong for a retail platform. The five missing items are institutional-grade nice-to-haves.

Tier 4 — Strategy lifecycle (the thing that retains users)

Capability	Status	Notes
Save strategy	☐	
Compare against past saves	☐	History tab
Fork existing strategy	☐	forked_from_id
Publish to community	☐	
Track live performance	☐	live_performance_service daily cron
Live signal computation (cron)	☐	signal_service.compute_current_signal
Signal change alerts (email)	☐	signal_alert.py + signal_jobs.py
Verified-author badges	☐	
Paper trading	☐	No broker integration
Live execution	☐	Same
Position reconciliation	☐	
Strategy versioning (v1, v2, v3 with diff)	☐	Each save is immutable; no lineage thread

Score: 8/12. The “tell me when the signal fires” loop is built, which is the single most-asked-for retention feature. Paper / live trading is the next frontier.

Tier 5 — Builder UX & education (where retail wins or loses)

Capability	Status	Notes
Template gallery with filters	☐	
Per-template detail page	☐	
5-question wizard	☐	
4-block summary editor	☐	
Risk preset → engine params	☐	
Multi-turn chat builder	☐	strategy_builder_iterate
Markdown research note → strategy	☐	parse_strategy_markdown
Concept explainer (define jargon)	☐	concept_explainer chat tool
Onboarding tutor	☐	onboarding_tutor chat tool
Result viewer with equity / DD / heatmap	☐	
Asset behavior fingerprint at picker time	☐	
Signal composer UI (custom WHEN IN / WHEN OUT)	☐	

Capability	Status	Notes
Per-primitive fire-rate report in results	☐	
Contribution attribution per trade	☐	
Custom Python signals (Pro tier)	☐	

Score: 10/15. Builder UX is mature for the template path; custom path needs the composer to graduate from “tuneable template” to “real custom”.

Tier 6 — Safety & operations

Capability	Status	Notes
Pre-backtest universe validation	☐	
Pre-backtest data availability check	☐	
Data quality gate	☐	
Tier-based caps (universe, history, runs/week)	☐	With shadow-mode logging chat_guardrails
Citation enforcement in chat	☐	
Refusal classifier (4 categories)	☐	
PostHog event capture	☐	
Weekly usage tracking	☐	
Survivorship-bias-free backtests	☐	
Cross-block consistency check (entry trend + exit MR)	☐	
Turnover-vs-cost realism check	☐	
template_usage_events analytics table	☐	

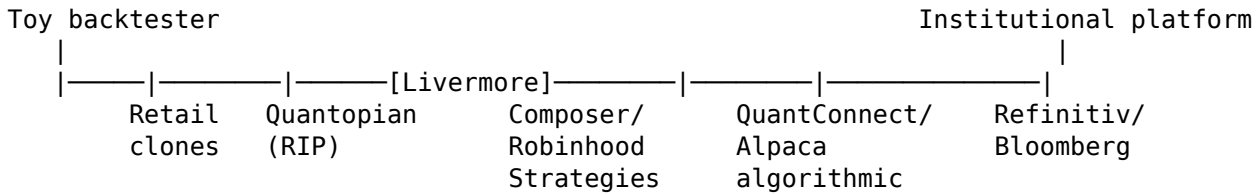
Score: 8/12. Safety net is solid for known failure modes; some quant-specific bias checks remain.

Total expert-lens score

Tier	Score	What it means
Tier 1 — Foundation	12/12	World-class
Tier 2 — Strategy expressiveness	9/13	Strong for long-only daily; missing composition + shorting
Tier 3 — Robustness	9/14	Above-average; gaps are nice-to-haves
Tier 4 — Lifecycle	8/12	Strong; paper trading is the frontier
Tier 5 — Builder UX	10/15	Templates great; custom needs the composer
Tier 6 — Safety	8/12	Strong baseline; bias prevention can deepen
Total	56/78 = 72%	Above the maturity threshold for a serious retail quant platform

6. Maturity positioning

Where Livermore sits on the platform-maturity spectrum:



Above Livermore on the curve, what's missing:

- Composable user-defined signal primitives in a sandboxed Python or DSL.
- Long/short equity with borrow cost modeling.
- Walk-forward optimization with automated parameter refitting.
- Paper trading integrated with a broker.
- Options strategies.
- Intraday signals.

Below Livermore, what you've already cleared:

- Naïve in-memory backtester with no tier controls.
- "Backtest only" platforms with no save/share/monitor loop.
- Platforms that show Sharpe without flagging credibility issues.
- Platforms with no survivorship/lookahead defense at all.
- Platforms whose chat-AI just makes up numbers without citation.

The two most impactful next investments:

1. **Signal composition layer** (Workstream A from the audit doc): unlocks the "real custom builder" promise and moves Livermore from "tunable templates" to "true custom strategies". ~11 engineering hours.
2. **Asset behavior fingerprint** (Workstream B item 1): a single small service (compute trending % and realized vol from price history) that improves both flows materially. Surface at Step 4 of "Use Template" and Step 2 of "Custom Build". ~3 engineering hours.

After these two: the signal-composition gap closes, and the asset-strategy fit warning loop closes. The platform graduates from "great retail quant tool" to "the obvious place a retail quant builds and monitors strategies".

7. Cross-references

- High-level audit: `framework/strategy_builder_audit.md`
- Risk-level spec: `framework/risk_control_prompt.md`
- Iteration framework (quarterly cycle): `framework/Livermore_Library_Iteration_Framework.html`
- Retail picker framework (interactive): `framework/Retail_Strategy_Picker_Framework.html`
- Strategy Library v2 (catalog + integration prompts): `Quant Strategy building/Livermore_Strategy_Library`

Deep-dive performed against main HEAD 4d5ace3 on 2026-05-26.